Lab 7: Full Stack Robotics: Perception, Planning, and Control *

EECS/ME C106A/206A Fall 2026

Goals

By the end of this lab, you should be able to:

- Detect an AR tag using a camera mounted on the UR7e
- Generate trajectories in the robot's workspace
- Implement and tune a workspace PID controller on trajectories
- Implement visual servoing to follow a moving AR tag

Contents

1 Introduction **2**

2 Theory **2**
- 2.1 Trajectories . . . 2
- 2.2 PID Control . . . 2
  - 2.2.1 Components of PID Control . . . 3
  - 2.2.2 Advantages of PID Control . . . 3
  - 2.2.3 Challenges . . . 3
  - 2.2.4 Tuning a PID Controller . . . 4
  - 2.2.5 Implementing the Derivative Term . . . 4
  - 2.2.6 Implementing the Integral Term . . . 4
- 2.3 Joint Space PID Control . . . 5
- 2.4 Workspace PID Control . . . 5
  - 2.4.1 The Control Law . . . 5
  - 2.4.2 The Jacobian . . . 5
- 2.5 Visual Servoing . . . 6

3 Your Tasks **7**
- 3.1 Perception: Static transformations for AR tags . . . 7
- 3.2 Planning: Defining Trajectories . . . 7
  - 3.2.1 Task 1: Linear Trajectory . . . 8
  - 3.2.2 Task 2: Circular Trajectory . . . 8
  - 3.2.3 Testing Trajectories . . . 8
  - 3.2.4 Demonstrating Trajectories with a Joint Space Controller . . . 8
- 3.3 Control: Workspace Setpoint Control . . . 9
- 3.4 Control: Workspace Trajectory Tracking . . . 10
  - 3.4.1 Tuning the Workspace Controller . . . 10
  - 3.4.2 The Effect of the Feedforward Input . . . 10
- 3.5 Control: Visual Servoing . . . 10

*Redesigned for Fall 2026 by Christopher-Michael Garcia, Reid Dye, and Valmik Prabhu. Based on previous work by Daniel Municio, Amay Saxena, Valmik Prabhu, and others

1 Introduction

The overall goal of this lab is to make the robot follow a moving AR tag in real time (visual servoing). To do this, you will have to develop a pipeline consisting of perception, planning, and control, and utilize much of the material you've learned in this class.

First, you'll implement some trajectories and learn the difference between workspace and joint space planning and control. Then you'll tune a workspace PID controller, first on setpoints, then trajectories, until ultimately its reliable enough to perform visual servoing.

2 Theory

2.1 Trajectories

Think of a path like a trail - it defines a set of states (positions) between the start state and goal state. In this metaphor, a trajectory would be the Strava recording of someone jogging along the trail. It contains the same position information, but it indexes it by time. A trajectory is a function that maps time t to a robot state. For our robot arm, this state can either be our joint angles, or our end effector pose.

A joint space trajectory defines a robot's trajectory as a function of its joint angles. It's defined as

$$\theta(t) = \begin{bmatrix} \theta_1(t) \\ \theta_2(t) \\ ... \\ \theta_n(t) \end{bmatrix} \tag{1}$$

where n is equal to the degrees of freedom of the robot. A workspace trajectory is defined as a function of the robot's pose in cartesian space. It's defined as the function

$$g_{ST}(t) = \begin{bmatrix} R_{ST}(t) & p_{ST}(t) \\ 0 & 1 \end{bmatrix} \tag{2}$$

To convert from a joint space to a workspace trajectory, you can use the twists of the robot and forward kinematics to derive the pose $g_{ST}(t)$ based off the joint angles at every time t. The other way around is a bit more challenging. For each pose, there could be multiple different solutions to the joint angles needed to make that pose (inverse kinematics), and it can be difficult to keep the resulting joint space trajectory smooth.

Defining tasks is often much more natural in the workspace (e.g., "move the end effector in a circle"), but when constraints like obstacles, joint limits, etc. are involved, usually you'll want to plan in joint space. This is why MoveIt plans in joint space.

2.2 PID Control

PID (Proportional–Integral–Derivative) control is one of the most widely used feedback control methods in robotics and industrial automation. Its popularity comes from its intuitive formulation and broad applicability.

A PID controller continuously computes the error between the desired and measured states, and then applies correctional controls based on three components. This PID video from MathWorks is very helpful.

The control law for a PID controller looks like

$$u(t) = u_{ff}(t) + K_p e(t) + K_i \int e(t)\,dt + K_d \frac{d}{dt} e(t). \tag{3}$$

Here $e(t)$ is the error at time t (which is the desired state minus the current state, $e(t) = x_d(t) - x(t)$), and u_{ff} is the *feedforward* input, which we will discuss below.

2.2.1 Components of PID Control

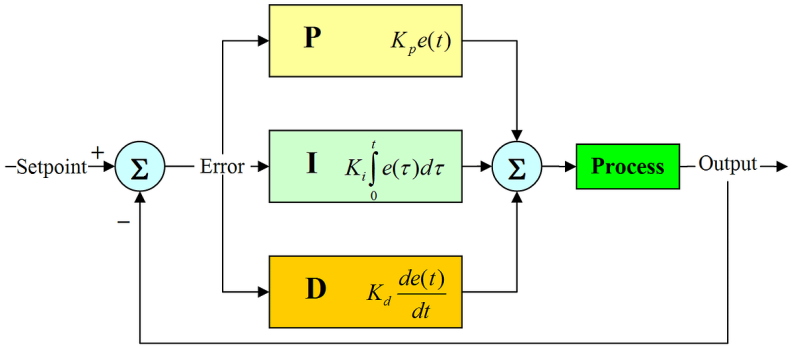


Figure 1: PID controller diagram

1. **Proportional (P)**: The proportional term produces an output that is proportional to the current error. The proportional response can be adjusted by multiplying the error by a constant known as K_p, the proportional gain constant.

$$P = K_p \times \text{error} \tag{4}$$

2. **Integral (I)**: The integral term is concerned with the accumulation of past errors. If the error has been present for an extended period of time, it will accumulate (integral of the error), and the controller will respond by changing the control output in relation to a constant K_i known as the integral gain.

$$I = K_i \times \int \text{error}\, dt \tag{5}$$

3. **Derivative (D)**: The derivative term is a prediction of future error, based on its rate of change. It provides a control output to counteract the rate of error change. The contribution of the derivative term to the overall control action is called the derivative gain, K_d.

$$D = K_d \times \frac{d(\text{error})}{dt} \tag{6}$$

4. **Feedforward (U_{ff})**: The feedforward, or target, input is often ignored when defining PID control, but it is perhaps the most important term. The feedforward input is the control input that *would* be required to follow your desired trajectory, if there were no disturbances and if all of your modeling assumptions were correct. A good way of thinking about this is that the feedforward term should handle everything you know about the system, and the P, I, and D terms should handle everything you don't.

The combined feedback output from all three terms is computed as:

$$\text{control} = U_{ff} + P + I + D \tag{7}$$

2.2.2 Advantages of PID Control

- Versatility: PID controllers can be used for a wide range of applications
- Stability: Properly tuned PID controllers can provide stable control for many processes
- Improved transient response: PID controllers can reduce the overshoot and settling time of a system

2.2.3 Challenges

- Requires tuning: The K_p, K_i, and K_d values need to be properly tuned for optimal performance, which can sometimes be a complex task
- Not suitable for all processes: Some systems might not benefit from one or more of the PID terms

2.2.4 Tuning a PID Controller

The default PID gains may not work too well right away. Tuning each gain has a different effect on the response of the controller.

In general, each gain has the following effect on control:

- Increasing K_p increases responsiveness
- Increasing K_i reduces steady-state error
- Increasing K_d reduces oscillations

First, start by increasing K_p until the state oscillates around the target. Next, increase K_d until the oscillations are sufficiently dampened. Then, increase K_i until steady-state error is reduced. Good tracking behavior should produce a small tracking error, smooth motion, and stable trajectories.

It is best practice to start tuning on more basic tasks and then progress in complexity once the gains work. For example, in this lab you'll first tune your gains on a static goal pose. Once that works, you'll then refine them on a trajectory that varies the goal pose over time. Finally, you will deploy your gains on the visual servoing task and do any final refinements.

2.2.5 Implementing the Derivative Term

Above, we defined the derivative term as $K_d \frac{d}{dt} e(t) = K_d(\dot{x}_d - \dot{x})$. This is our derivative gain times the difference between the derivative of our trajectory, and the derivative of our state. But there are two problems here

Firstly, if we're controlling the velocity of the system like we do in this lab, the derivative term doesn't actually do much. $\dot{x}$ is our control input and $\dot{x}_d$ is our feedforward input and their difference is just the proportional and integral terms. Instead, we define a different derivative term:

$$D(t) = K_d \frac{e(t) - e(t - \delta t)}{\delta t} \tag{8}$$

This smooths the input over time and makes the control less jerky or oscillatory.

Secondly, the derivative term can act as a high pass filter, since it directly multiplies the derivative. Usually, sensing the derivative is pretty noisy, and the derivative term can amplify this noise. In practice, you usually want to apply a low-pass filter on the error derivative before passing it into the derivative controller. But in this lab, this isn't necessary.

2.2.6 Implementing the Integral Term

Above, we defined the integral term as $K_i \int e(t)dt$. But how do we actually compute this? In practice, since we usually apply our controller at discrete time intervals (discrete time control), we can approximate this integral as a sum:

$$I(t) = K_i e_{int}(t - \delta t) \tag{9}$$

$$e_{int}(t) = e_{int}(t - \delta t) + e(t)\delta t \tag{10}$$

However, if the integral is high for a prolonged time (for example, you hold the robot back with your hand) the integral term can become very large. This is called windup. When the proverbial hand is removed, it will take a lot of negative error to bring the integral term down again, and this can cause disasterous motion. To mitigate this issue, we usually want to limit the growth of the integral term. This is called anti-windup. One method of doing this is with a discount term

$$e_{int}(t) = \alpha\, e_{int}(t - \delta t) + e(t)\delta t, \quad \alpha < 1 \tag{11}$$

This exponentially discounts past error relative to current error, so the integrator term cannot wind up infinitely.

2.3 Joint Space PID Control

When controlling the robots joints directly, we essentially apply an individual PID controller on each joint. At each timestep, we perform

$$e = \theta_d - \theta \tag{12}$$

$$\dot{\theta} = \dot{\theta}_d + K_p e + K_d \frac{(e - e_{prev})}{\delta t} + K_i e_{int} \tag{13}$$

$$e_{int} = \alpha\, e_{int} + e\, \delta t \tag{14}$$

This is nice because each joint is has its own mechanical idiosyncrasies, and you can compensate for them individually with the control gains.

The disadvantage is that there are a lot of gains to tune. For a UR7e, you would need to tune 18 different gains, plus potentially another 6 low-pass filters and another 6 anti-windup discount terms.

2.4 Workspace PID Control

Controlling the end effector's position and orientation at the same time involves a PID Controller with an error term that tracks the error in pose. This algorithm is very similar to the Jacobian inverse kinematics algorithm, so much so that workspace velocity control is sometimes just called inverse kinematics.

2.4.1 The Control Law

We can take our control input as the end effector's spatial velocity $V^s \in \mathbb{R}^6$.

From lecture, the spatial configuration error is $g_d g^{-1}$. The exponential coordinates of this error is the spatial velocity needed to move from g to g_d. We extract this using the log operator and then the vee.

$$\xi_e = log(g_d g^{-1})^\vee \tag{15}$$

Once we have our error term, we can create a control law. At each timestep, we apply:

$$V_s = V_{ff}^s + K_p \xi_e + K_d(\xi_e - \xi_{e_{prev}})/\delta t + K_i(\xi_{e_{int}}) \tag{16}$$

Note that unlike joint-space control, we only have one K_p, one K_i, and one K_d. Ie K_p, K_i, $K_d \in \mathbb{R}$, so you only have three numbers to tune.

V_{ff}^s is the feed-forward velocity that comes from the trajectory itself, not the current pose error. It's defined as

$$V_{ff}^s = \dot{g}_d g_d^{-1} \tag{17}$$

Putting everything together, at each timestep we execute

$$\xi_e = log(g_d g^{-1})^\vee \tag{18}$$

$$V_{ff}^s = (\dot{g}_d g_d^{-1})^\vee \tag{19}$$

$$V_s = V_{ff}^s + K_p \xi_e + K_d(\xi_e - \xi_{e_{prev}})/\delta t + K_i(\xi_{e_{int}}) \tag{20}$$

$$\xi_{e_{int}} = \alpha\, \xi_{e_{int}} + \xi_e\, \delta t \tag{21}$$

2.4.2 The Jacobian

From lecture, the spatial Jacobian matrix can encode joint velocities $\dot{\theta}$ into a spatial velocity $V^s \in \mathbb{R}^6$ by using the formula:

$$V^s = J^s(\theta)\dot{\theta} \tag{22}$$

The Jacobian is dependent on each joint position in the kinematic chain, so it must be updated each time the robot moves. We know what V_s is, so we need to solve for the joint velocities $\dot{\theta}$. To do that, we need to multiply the spatial velocity by the pseudo inverse spatial Jacobian. Thus, our working joint velocities are

$$\dot{\theta} = J^\dagger(\theta) V_s \tag{23}$$

And this is what you send to the robot. It publishes the joint velocities on the topic
`/forward_velocity_controller/commands`

2.5 Visual Servoing

Visual servoing is when you control your robot using real-time camera information. In this lab, our goal is to follow a moving AR tag. At each timestep, the camera gives us the pose of the AR tag relative to the robot, and thus our desired pose. However, since we don't know how the AR tag will move in the future, we cannot define a trajectory to follow. In addition, inverse kinematics takes too long to reliably run at each timestep, so we must do our control in the workspace.

Note that since we cannot define a trajectory $\dot{g}_d = 0$, so $V_{ff}^s = 0$ for visual servoing

3 Your Tasks

Before beginning this lab, make sure to run

```
git pull starter main
```

in `ros_workspaces` to pull the Lab 7 starter code.

Replace the `kin_func_skeleton` and `jacobian_func_skeleton` in the starter code with your own implementations. **Make sure to fully replace the entire starter files with your own implementations.**

3.1 Perception: Static transformations for AR tags

TASK: Recall the static transform broadcaster you used in lab 5. We will use another static transform broadcaster to broadcast the transformations between the camera and wrist link, but we're going to add a transformation for a goal pose relative to each two AR tags (one big and small). These goal poses will be used for the rest of the lab, because we're going to try and move the end effector to them.

- The transformation `G2` creates a pose above the small AR tag (the one on a stick) such that when the end-effector moves to the pose, the camera will be 30 cm above it. We will use this goal pose for visual servoing.
- The transformation `G3` creates a pose above the large AR tag such that when the end-effector moves to the pose, it will be 50 cm above it. Unlike the small AR tag, this tag can lie flat on the table. Thus, we can use this goal pose when trying to run the workspace controller on a setpoint or trajectory.

Go to the lab supplies table and choose an AR tag on a tripod. You already have a larger AR tag on the desk below the robot. In `static_tf_transform.py`, enter the marker IDs for **both** AR tags **as a string** in the format `ar_marker_<your number>`. **You MUST do this BEFORE continuing with the lab and running anything, or else, you will get errors.**

Run the following commands in **separate terminals** to start all the relevant components of our robotics pipeline.

```
ros2 run ur7e_utils enable_comms
ros2 launch visual_servoing lab7.launch.py
```

We encourage you to inspect `lab7.launch.py` to see which nodes are being launched.

Now that RViz is launched, add the following displays:

- Click **Add** → **By display type** → **TF**
- Click **Add** → **By topic** → `/camera/camera/color/image_raw` → **Image**

Verify that the AR tag frame and goal frame appears in the TF tree for each tag.

3.2 Planning: Defining Trajectories

Once the AR tag pose is known, the robot will generate a trajectory with respect to the AR tag (ex: draw a vertical line above it).

TASK: You will implement two trajectory classes in `trajectories.py`:

- LinearTrajectory
- CircularTrajectory

Each trajectory must implement the `target_pose` and `target_velocity` methods.

`target_pose` takes in time and returns a position and quaternion for the end effector. `target_velocity` takes in time and returns the end effector's spatial velocity. Note that since both of these trajectories have a constant end effector orientation,

$$\dot{g}_d = \begin{bmatrix} 0 & \dot{p} \\ 0 & 0 \end{bmatrix}, \quad \text{so} \quad V_{ff}^s = \begin{bmatrix} \dot{p} \\ 0 \end{bmatrix}$$

3.2.1 Task 1: Linear Trajectory

The robot should **translate** the end effector from

$$\text{start_position} \rightarrow \text{goal_position}$$

using a trapezoidal velocity profile. This profile accelerates with constant acceleration a during the first half of the motion and decelerates with constant acceleration $-a$ during the second half. The following kinematic equations may help:

$$s(t) = \frac{1}{2}at^2 \tag{24}$$

$$v(t) = at \tag{25}$$

Your implementation should compute both the desired position and velocity at time t to move from the start position to goal position. The constructor for each class defines variables that may be helpful for your implementation. Also, remember that these formulas need to be vectorized because the kinematic equations apply for the X, Y, and Z components of the translation vector.

3.2.2 Task 2: Circular Trajectory

The robot traces a circular trajectory centered at the AR tag, customized by the following parameters:

- center_position
- radius
- total_time

You may find the following equations helpful for this task:

$$\theta(t) = \frac{1}{2}\alpha t^2 \tag{26}$$

$$\dot{\theta}(t) = \alpha t \tag{27}$$

$$x = x_c + r\cos(\theta) \tag{28}$$

$$y = y_c + r\sin(\theta) \tag{29}$$

$$\dot{x} = -r\dot{\theta}\sin(\theta) \tag{30}$$

$$\dot{y} = r\dot{\theta}\cos(\theta) \tag{31}$$

3.2.3 Testing Trajectories

You can visualize trajectories without running the robot.

```
python3 trajectories.py --task line --animate
python3 trajectories.py --task circle --animate
```

Verify that the:

- trajectory begins and ends at the correct positions
- velocity is continuous
- position is differentiable
- linear path is straight
- circular path forms a complete circle

3.2.4 Demonstrating Trajectories with a Joint Space Controller

You can see your trajectories working using MoveIt's joint space velocity controller. We've implemented a converter that repeatedly runs inverse kinematics to turn a workspace trajectory into a joint space trajectory.

The MoveIt controller uses the UR controller action server `control_msgs/action/FollowJointTrajectory` to execute trajectories. As a refresher, action servers offer the following the following benefits:

- asynchronous execution
- feedback during execution
- cancellation

To execute the trajectories that you implemented in the previous tasks with the joint space controller, you can use the following command:

```
ros2 run visual_servoing JointTrajectory --task line --ar_marker <your marker>
```

The `line` task commands the robot to trace a vertical line above the AR tag, while the `circle` task commands the robot to trace a horizontal circle centered at the AR tag. If the trajectories take too long to run (or the robot doesn't appear to move much), you can reduce the trajectory time at the bottom of `JointTrajectory.py`.

Please take a moment to read through `JointTrajectory.py` and understand how it moves the robot arm along both the trajectories.

3.3 Control: Workspace Setpoint Control

Now let's head into `main.py`. It is very similar to `JointTrajectory.py`, but this controller uses the workspace control law instead of the joint space one.

We're going to make the workspace controller move the end effector to a setpose above the AR marker (which is being broadcasted by `static_tf_transforms.py`).

First fill out the `find_error` function, which calculates ξ_e from the current and desired poses. *Hint: Use the functions you defined in `jacobian_func_skeleton`.*

Then fill out the `workspace_controller` function to implement the formulas in (18)-(21). In it, you must:

- calculate the pose error between the current and goal pose using `find_error()`
- Use the workspace control law to calculate the needed spatial velocity
- Update the integrator term

Note: The code defines the error as η rather than ξ_e, and the integral as ν rather than $\xi_{e_{int}}$

Once all of that is done, and you AR tag is visible on Rviz (IMPORTANT: make sure the lettering is facing the computer so the pose orientation is correct), you can run your controller on a setpose using

```
ros2 run visual_servoing main --task pose --controller pid --ar_marker <your marker>
```

Tune your K_p value found at line 67 to make the end effector reach the pose without overshooting too much or taking too long. Leave K_i, $K_d = 0$.

Checkpoint 1

Submit a checkoff request at `https://tinyurl.com/ee106a-fa26lab`. At this point you should be able to demonstrate:

- successful AR tag detection in RViz
- successful linear and circular trajectory execution with the joint space controller
- successful execution with the workspace controller to a setpoint above the AR tag
- the difference between joint space and workspace controllers in this example.
- how you tuned Kp for the workspace controller.
- what is V_{ff} for the pose trajectory? Why?

3.4 Control: Workspace Trajectory Tracking

For this section, you will continue working in `main.py`. We will make the robot follow a trajectory using the workspace controller. You have the control law and trajectories already defined from before. All you need to do is to define the points of reference.

Task 1: In the `VisualServo` node, fill out the functions `make_line()` and `make_circle()` to instantiate the trajectories that the end effector is going to follow. For `make_line()`, set the starting point as the current position of the end effector and the target as the pose generated above the AR tag (find out what it's called). For `make_circle()`, set the goal pose for you used for `make_line()` as the center and the radius as an argument to pass in.

3.4.1 Tuning the Workspace Controller

You can test your workspace controller using the following command (change 'line' to 'circle' for circular trajectory).

```
ros2 run visual_servoing main --task line --controller pid --ar_marker <your marker> \
--total_time 20 --circle_radius 0.05
```

Safety Notice: Keep your hand on the **emergency stop** at all times! Do not execute unless workspace is clear and transforms are verified.

Task 2: Tune your PID controller. Add in the derivative term first, and then the integral, until the robot follows the trajectory smoothly.

3.4.2 The Effect of the Feedforward Input

Task 3: Temporarily edit your workspace controller to set $V_{ff}^{s} = 0$ (make sure it's the correct shape!). How does your controller's performance change? Make sure to undo your change when you're done,

3.5 Control: Visual Servoing

Now that we have our robot smoothly following a linear and circular trajectory using a workspace controller, we are going to have it follow a moving AR tag. Take a stick with a small AR tag attached to it from the lab supplies and hold it under the camera. Look at the goal pose it generates above the tag; that is the goal pose of our end effector.

Task 1: Implement the control loop inside `run_visual_servoing()`. Use everything you've learned about workspace control theory to convert the error between the desired and current pose into `joint_speeds`.

Run it with

```
ros2 run visual_servoing main --controller pid --ar_marker <your marker>
```

Checkpoint 2

Submit a checkoff request at `https://tinyurl.com/ee106a-fa26lab`. At this point you should be able to:

- show your workspace controller implementation for trajectories (line and circle) and visual servoing
- explain how you tuned the PID gains
- explain the effect of feedforward velocity when following a trajectory, and why we can't use it for visual servoing
- explain how you converted the pose error into joint velocities for visual servoing

---